

ASP DOT NET CORE API

by MUHAMMAD FARHAN

What is API?

An API (Application Programming Interface) is a set of rules and protocols that allows different software applications to communicate with each other. It defines how requests are made and how responses are returned between systems.

Key Points

- Acts as a **bridge** between two applications
- Allows one system to **access features or data** of another system
- Hides internal implementation (only exposes necessary parts)
- Can be used for:
 - Data sharing
 - Service integration
 - Automation
- Works using:
 - Functions (in libraries)
 - Endpoints (in web APIs)

Types of APIs

- **Web APIs** → communicate over HTTP
- **Library APIs** → used within programming languages
- **Third-party APIs** → provided by external services

What is Web API?

A Web API (Application Programming Interface) is a service that allows communication between different applications over the web using standard protocols like **HTTP**. It enables clients (such as browsers, mobile apps, or other services) to request and exchange data with a server.

Key Points

- Uses **HTTP methods** like:
 - GET → retrieve data
 - POST → create data
 - PUT → update data
 - DELETE → remove data
- Data is usually transferred in:
 - **JSON** (most common)
 - XML (less common)
- It follows the **client-server architecture**

- Can be consumed by:
 - Web applications
 - Mobile apps
 - Desktop applications
 - Other APIs

How it Works:

1. Client sends an HTTP request to the API endpoint
2. Server processes the request
3. Server returns a response (usually JSON data)

Example flow:

- Client: GET /api/products
- Server: Returns list of products in JSON

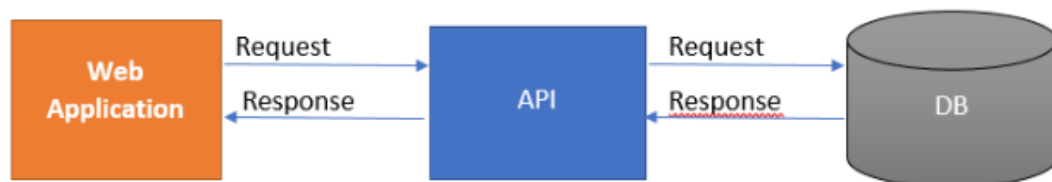
What is Asp.Net API?

An **ASP.NET API (ASP.NET Web API)** is a framework provided by ASP.NET Core that is used to build **RESTful web services**. It allows developers to create APIs that can be accessed over HTTP by different clients such as web apps, mobile apps, and other services.

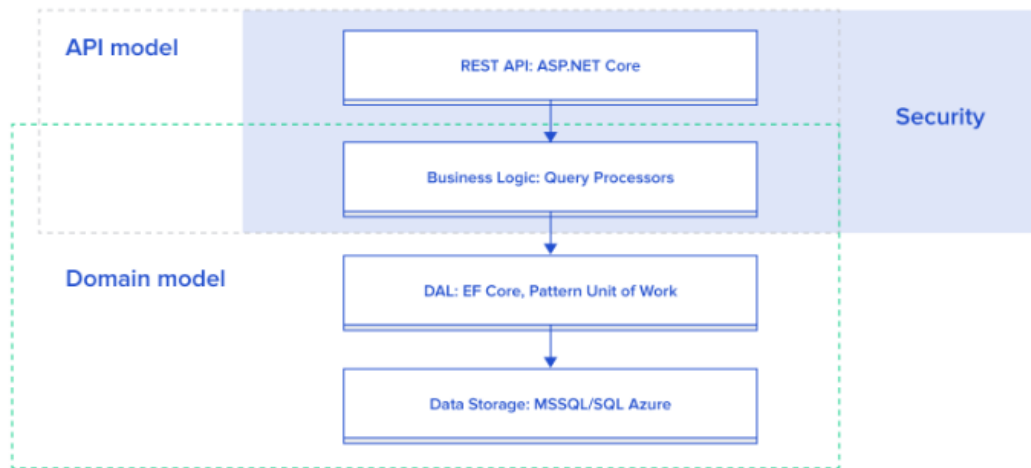
These API Use Many Places:

- Browsers
- Mobile Applications
- Desktop Applications
- IOT's

API stands for Application Programming Interface. It is an intermediate software agent that allows two or more applications to interact with each other.



API LAYER:



What are RESTful Services?

RESTful Services are web services that follow the principles of **REST (Representational State Transfer)**. They use standard HTTP methods to perform operations on resources and communicate over the web in a stateless manner.

REST FUL Constraint:

- **Client Server Architecture** - separation of concerns
- **Stateless** - each request is independent
- **Cacheable** - responses can be cached
- **Uniform interface** (HTTP VERBS)
- **Layered System** can have multiple layers (security, load balancing)

HTTP VERBS:

GET:

The Get method is used to retrieve data.

Example: Get the book data, search a book, etc.

POST:

The POST method is used to make a new entry in the database.

Example: Add a new student, etc.

PUT:

The Put method is used to update all the properties of current resource in the database.

Example: Update all properties (columns) of an existing student, etc.

PATCH:

The Patch method is similar to Put but it is used to update partial properties of current resource in the database.

Example: Update few properties(columns) of an existing student, etc.

DELETE:

The Delete method is used to delete the resource from the database.

Example: Delete an existing student from database, etc.

Response Status codes:

- 1xx informational response
- 2xx successful
- 3xx Redirection
- 4xx Client error
- 5xx Server error

Some Actual Code:

- **200** OK
- **201** New Resource created
- **204** No content (The server processed the request successfully and it is not returning any content)
- **301** Moved permanently
- **302** Moved temporarily
- **400** Bad request
- **401** Unauthorized
- **404** Not found

Software should be installed:

- Visual Studio
- SQL Server
- Postman

Example of Simple API:

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet("{id}")]
    public IActionResult Get(int id)
    {
        return Ok(new { Id = id, Name = "Product" });
    }
}
```

Simple ASP.NET Core Web API (CRUD Example)

Model (Product.cs)

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Controller (ProductsController.cs)

```
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    // Fake in-memory data
    private static List<Product> products = new List<Product>
    {
```

```
    new Product { Id = 1, Name = "Laptop", Price = 50000 },  
    new Product { Id = 2, Name = "Mobile", Price = 30000 }  
};
```

```
// GET: api/products
```

```
[HttpGet]
```

```
public IActionResult GetAll()
```

```
{  
    return Ok(products);  
}
```

```
// GET: api/products/1
```

```
[HttpGet("{id}")]
```

```
public IActionResult GetById(int id)
```

```
{  
    var product = products.FirstOrDefault(x => x.Id == id);  
    if (product == null)  
        return NotFound();  
  
    return Ok(product);  
}
```

```
// POST: api/products
```

```
[HttpPost]
```

```
public IActionResult Create(Product product)
```

```
{  
    products.Add(product);  
    return Ok(products);  
}
```

```

// PUT: api/products/1
[HttpPut("{id}")]
public IActionResult Update(int id, Product updatedProduct)
{
    var product = products.FirstOrDefault(x => x.Id == id);
    if (product == null)
        return NotFound();

    product.Name = updatedProduct.Name;
    product.Price = updatedProduct.Price;

    return Ok(product);
}

// DELETE: api/products/1
[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    var product = products.FirstOrDefault(x => x.Id == id);
    if (product == null)
        return NotFound();

    products.Remove(product);

    return Ok(products);
}
}

```

Note: you can same work with database table using entity framework.